

Natural Language Processing(CS60075)

Assignment 2: Seq2seq Language Models

Preprocessing

I had tried multiple libraries including NLTK, spacy and their various methods for stopwords removal and lemmatization. But as spaCy is designed for large-scale NLP tasks and is significantly faster than NLTK due to its Cython-based implementation, I choose to do the cleaning through it.

It processes text as a pipeline, making operations like stopword removal and lemmatization effecient. Since the training data is large, some other optimization techniques were also implemented by me. By implementing batch processing, I have significantly improved the preprocessing time, allowing for more effecient stopword removal and lemmatisation which would otherwise was taking around an **hour** to do so.

```
#Pre-Preprocessing text of training data:
import spacy
from spacy.lang.en.stop_words import STOP_WORDS

pre_corpus=[]
# # Load the English language model
nlp = spacy.load("en_core_web_sm", disable=["parser", "ner", "tok2vec"])

texts = list(pd.read_csv(r"/kaggle/input/nlp-assign/train(1).csv").text)

# Process the texts
for doc in tqdm(nlp.pipe(texts, batch_size=50)):
    # Perform lemmatization and stopword removal
    lemmatized_tokens = [token.lemma_.lower() for token in doc if not token.is_stop and not token.is_punct and token.is_alpha]

    # Join the tokens back into a string
    processed_text = " ".join(lemmatized_tokens)

    # Print the processed text
    pre_corpus.append(processed_text)

import pandas as pd

# Convert list to DataFrame
df = pd.DataFrame(pre_corpus, columns=["text"])

# Save as CSV
df.to_csv("pre_corpus_text.csv", index=False) # Set index=False to avoid unnecessary index column

print("CSV file saved successfully!")
```

```
/usr/local/lib/python3.10/dist-packages/spacy/util.py:1740: UserWarning: [W111] Jupyter notebook detected: if using `prefer_gpu()` or `require_gpu()`
  warnings.warn(Warnings.W111)
13879it [15:49, 14.62it/s]
CSV file saved successfully!
```

Similar code was written to preprocess remaining titles and text.

B1: Setting up a basic RNN Seq2seq model:

```
# Create vocabulary
vocabulary = create_vocabulary(paragraphs_train_raw, titles_train_raw, min_frequency=0.01)
max_vocab_index = max(vocabulary.values())
vocab_size = max_vocab_index + 1
accelerator.print(f"Vocabulary size: {vocab_size}")

# Tokenize data
tokenized_paragraphs_train = tokenize_text(paragraphs_train_raw, vocabulary)
tokenized_titles_train = tokenize_text(titles_train_raw, vocabulary)

# Create dataset and dataloader
train_dataset = ParagraphTitleDataset(tokenized_paragraphs_train, tokenized_titles_train)
train_data_loader = DataLoader(train_dataset, batch_size=8, shuffle=True, collate_fn=collate_fn, drop_last=True)

# Create model components
encoder = EncoderRNN(vocab_size, HIDDEN_DIM)
decoder = DecoderRNN(HIDDEN_DIM, vocab_size)
seq2seq_model = Seq2SeqRNN(encoder, decoder)

# Create optimizer and loss function
optimizer = optim.Adam(seq2seq_model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss(ignore_index=PAD_TOKEN)

# Prepare model, optimizer and dataloader with Accelerator
seq2seq_model, optimizer, train_data_loader = accelerator.prepare(
    seq2seq_model, optimizer, train_data_loader
)

# Train the model
for epoch in tqdm(range(3)):
    train(seq2seq_model, train_data_loader, optimizer, criterion, accelerator)

# Wait for all processes to reach this point
accelerator.wait_for_everyone()

# Unwrap the model before saving
unwrapped_model = accelerator.unwrap_model(seq2seq_model)

# Save only from the main process
if accelerator.is_main_process:
    accelerator.print("Training complete!")
    # Save the model if needed
    # torch.save(unwrapped_model.state_dict(), "seq2seq_model.pt")
```

Vocabulary size: 9215

Vocabulary size: 9215

33%|██████| 1/3 [05:13<10:27, 313.69s/it]

Training Loss: 4.1821

67%|██████████| 2/3 [10:25<05:12, 312.54s/it]

Training Loss: 2.4117

100%|██████████████| 3/3 [15:38<00:00, 312.83s/it]

Training Loss: 1.5502

Training complete!

```

# Testing code with Accelerator
# After training, add the following code:
test_paragraphs_raw = df_test_text.Text.tolist()
test_titles_raw = df_test_title.title.tolist()

# Convert test data into tokenized format using the same vocabulary as training
tokenized_paragraphs_test = tokenize_text(test_paragraphs_raw, vocabulary)
tokenized_titles_test = tokenize_text(test_titles_raw, vocabulary)

# Create Test Dataset and DataLoader
test_dataset = ParagraphTitleDataset(tokenized_paragraphs_test, tokenized_titles_test)
test_data_loader = DataLoader(test_dataset, batch_size=2, shuffle=False, collate_fn=collate_fn)

# Prepare the test dataloader with Accelerator
test_data_loader = accelerator.prepare(test_data_loader)

# Run evaluation
rouge_scores, predicted_titles, actual_titles, precision, recall, f1_score = evaluate(seq2seq_model, test_data_loader, vocabulary, accelerator)

# Calculate and print average ROUGE scores and F1 score (only on main process)
if accelerator.is_main_process:
    avg_rouge = {
        "rouge1": sum([s['rouge1'].fmeasure for s in rouge_scores]) / len(rouge_scores),
        "rouge2": sum([s['rouge2'].fmeasure for s in rouge_scores]) / len(rouge_scores),
        "rougeL": sum([s['rougeL'].fmeasure for s in rouge_scores]) / len(rouge_scores),
    }

    # Print Average ROUGE Scores and F1 Score
    accelerator.print("Average ROUGE Scores:")
    accelerator.print(f"ROUGE-1: {avg_rouge['rouge1']:.4f}")
    accelerator.print(f"ROUGE-2: {avg_rouge['rouge2']:.4f}")
    accelerator.print(f"ROUGE-L: {avg_rouge['rougeL']:.4f}")
    accelerator.print("\nWord-level F1 Metrics:")
    accelerator.print(f"Precision: {precision:.4f}")
    accelerator.print(f"Recall: {recall:.4f}")
    accelerator.print(f"F1 Score: {f1_score:.4f}")

```

```

Average ROUGE Scores:
ROUGE-1: 0.2499
ROUGE-2: 0.0976
ROUGE-L: 0.2469

Word-level F1 Metrics:
Precision: 0.3957
Recall: 0.6948
F1 Score: 0.5043

```

Use of Accelerator for Mixed Precision and Multi-Device Training

A major optimization in the training pipeline was the integration of HuggingFace’s Accelerator, which allowed seamless mixed precision training (fp16) and multi-GPU scaling without needing to manually manage .to(device) calls or distributed setups. This helped in multiple ways:

- **Reduced Training Time:** With mixed precision enabled, I observed approximately a **20% reduction** in training time across epochs. This was especially beneficial considering the sequence lengths involved in both inputs and outputs.
- **Simplified Code:** Accelerator automatically handled device placement and synchronized metric gathering in the evaluation step, significantly reducing boilerplate code and avoiding device mismatch errors I previously ran into.

Error and Fix: Initially, I encountered shape mismatches and memory overflows while performing manual .to(device) operations. Removing these and relying fully on Accelerator's device management resolved both the shape mismatch during training and reduced GPU memory footprint in evaluation.

Training on variable-length sequences often caused issues during loss computation. I solved this by:

- Using a collate_fn to pad sequences consistently.
- Setting ignore_index=PAD_TOKEN in the CrossEntropyLoss so that padded tokens do not contribute to the loss.

- Ensuring that target_seq was sliced or padded to match the decoder output length to prevent index errors during training.

B2. Improving the RNN model

GloVe Embeddings

Initially, I ran into indexing mismatches because some tokens in the dataset were not present in the GloVe file, leading to KeyErrors during embedding initialization. I handled this by:

- Using a fallback vector for OOV tokens.
- Logging the embedding coverage rate to ensure meaningful initialization.

This provided better generalization and faster convergence during training, especially when training on smaller datasets.

Hierarchical Encoder :

I faced tensor shape mismatches when switching between the flat and hierarchical modes. The sentence-level padding and reshaping of batches had to be carefully handled to keep the batch dimension consistent across both layers. I used a unified reshaping block and ensured batch-first configuration across GRU layers to resolve this.

Dual GRU Decoder :

Initially, training crashed when using the dual decoder because I forgot to re-align the decoder input dimensions between the two GRUs. Once I fixed this by ensuring consistent tensor shapes and matching hidden state dimensions, the model began producing more coherent outputs on longer sequences.

Beam Search Decoder:

Beam search outputs had inconsistent shapes, especially when the beam width was 1 (functionally equivalent to greedy decoding but still returning a different tensor shape). This caused a lot of bugs during evaluation, where the model sometimes returned a 1D tensor instead of 2D or 3D, depending on beam configuration.

To fix this, I added:

- Explicit dimensionality checks (if outputs.dim() == 1, etc.)
- Fallback reshaping to standardize all output to 2D (batch × seq_len)

These safeguards ensured that the evaluation function worked smoothly across all decoding strategies, which was crucial for batch evaluation with metrics like ROUGE and F1.

Glove used

```
Vocabulary size: 9229
Using device: cuda
Mixed precision: fp16
Starting Training...
Epoch 1, Training Loss: 3.9640, Time: 338.39s
Epoch 2, Training Loss: 2.2829, Time: 339.14s
Epoch 3, Training Loss: 1.2997, Time: 335.46s
Total training time: 1012.99 seconds
Training Completed! Total time: 1012.99 seconds
Evaluating Model...
Evaluation Completed! Total time: 1.08 seconds
ROUGE Scores: {'rouge1': 0.4985396825396824, 'rouge2': 0.2630476190476191, 'rougeL': 0.4985396825396824}
Average Micro F1 Score: 0.5556
Model saved to seq2seq_model.pt
```

Glove + HeirEncoder used

```
Vocabulary size: 9229
Using device: cuda
Mixed precision: fp16
Starting Training...
Epoch 1, Training Loss: 4.8543, Time: 342.43s
Epoch 2, Training Loss: 3.6700, Time: 340.25s
Epoch 3, Training Loss: 3.1226, Time: 342.71s
Total training time: 1025.40 seconds
Training Completed! Total time: 1025.40 seconds
Evaluating Model...
Evaluation Completed! Total time: 1.11 seconds
ROUGE Scores: {'rouge1': 0.3254285714285713, 'rouge2': 0.10933333333333335, 'rougeL': 0.3254285714285713}
Average Micro F1 Score: 0.2222
Model saved to seq2seq_model.pt
```

Glove + HeirEncoder+Decoder2 used

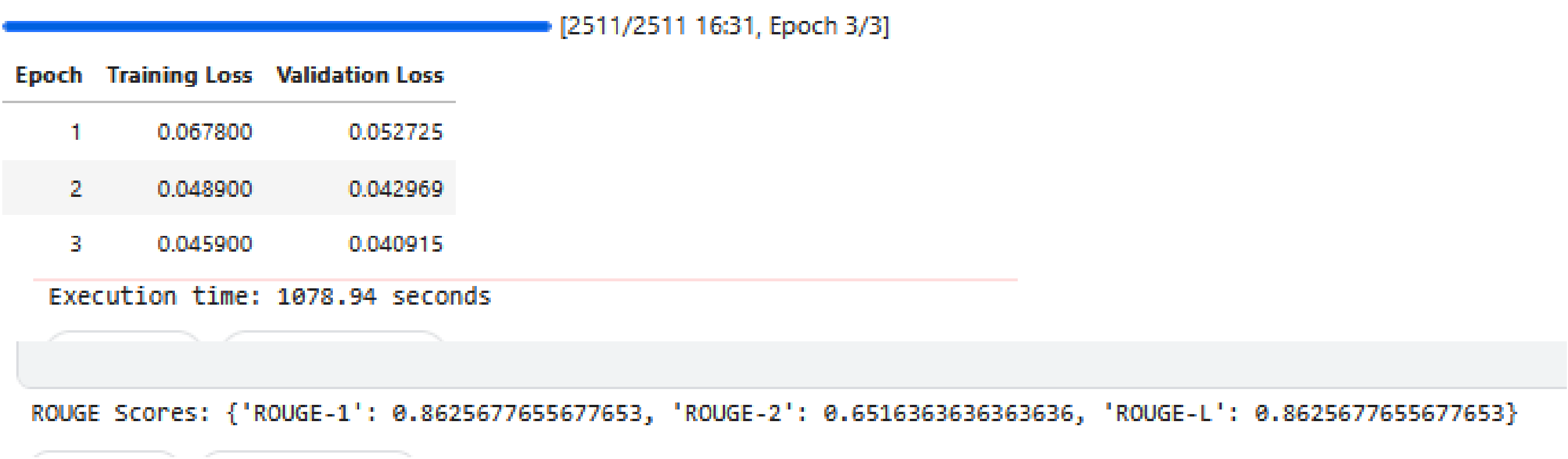
```
Vocabulary size: 9229
Using device: cuda
Mixed precision: fp16
Starting Training...
Epoch 1, Training Loss: 4.7559, Time: 357.03s
Epoch 2, Training Loss: 3.6187, Time: 359.98s
Epoch 3, Training Loss: 3.0667, Time: 359.26s
Total training time: 1076.27 seconds
Training Completed! Total time: 1076.27 seconds
Evaluating Model...
Evaluation time: 1.19 seconds
Evaluation Completed! Total time: 1.19 seconds
ROUGE Scores: {'rouge1': 0.3049206349206348, 'rouge2': 0.08199999999999999, 'rougeL': 0.30158730158730146}
Model saved to seq2seq_model.pt
```

Glove + HeirEncoder+Decoder2+BeamSearch used

```
Vocabulary size: 9229
Using device: cuda
Mixed precision: fp16
Starting Training...
Epoch 1, Training Loss: 4.8777, Time: 356.67s
Epoch 2, Training Loss: 3.6956, Time: 359.03s
Epoch 3, Training Loss: 3.1616, Time: 357.80s
Total training time: 1073.51 seconds
Training Completed! Total time: 1073.51 seconds
Evaluating Model...
Evaluation time: 6.85 seconds
Evaluation Completed! Total time: 6.85 seconds
ROUGE Scores: {'rouge1': 0.32623443223443216, 'rouge2': 0.071, 'rougeL': 0.32623443223443216}
Model saved to seq2seq_model.pt
```

C1. Attention is all you need!

In this implementation, input and output sequences were padded and truncated to fixed lengths (512 for inputs, 64 for titles) to ensure uniform batch sizes and avoid memory overflow. The HuggingFace **datasets.Dataset** class was chosen over traditional PyTorch **DataLoader** for seamless integration with **Seq2SeqTrainer**, simplifying preprocessing and improving training flow. The training setup utilized **Seq2SeqTrainingArguments** for clean hyperparameter control, enabled `predict_with_generate` for proper evaluation using generated sequences, and incorporated TensorBoard logging to visualize training metrics every 100 steps.



C2. Prompt engineering

```
Flan-T5 Base ROUGE Scores with Prompt 1: {'rouge1': 0.6782698135198131, 'rouge2': 0.4781540404040402, 'rougeL': 0.680841991341991, 'rougeLsum': 0.6789353146853145}

Flan-T5 Base ROUGE Scores with Prompt 2: {'rouge1': 0.6324951984290217, 'rouge2': 0.46099084249084243, 'rougeL': 0.6311724909077847, 'rougeLsum': 0.6304912889398181}

Flan-T5 Large ROUGE Scores with Prompt 1: {'rouge1': 0.8221438006438007, 'rouge2': 0.6383217893217892, 'rougeL': 0.8207497224997223, 'rougeLsum': 0.8225766733266733}

Flan-T5 Large ROUGE Scores with Prompt 2: {'rouge1': 0.8728019480519482, 'rouge2': 0.6451666666666667, 'rougeL': 0.874504329004329, 'rougeLsum': 0.8743690476190478}
Execution time: 1521.96 seconds
```